Project Overview

Local RAG-Based PDF Question Answering System

Project purpose

This project is a Retrieval-Augmented Generation (RAG) application that allows a user to upload a PDF and ask questions about its contents. Instead of relying only on the language model's general knowledge, the system retrieves relevant passages from the uploaded document and provides them to the generation model as context.

Core technology stack

Layer	Technology	Purpose
Frontend	React	PDF upload, question input, and answer/source display
Backend	FastAPI / Python	API endpoints and RAG pipeline orchestration
PDF processing	PyPDF	Extract text from PDF pages
Embeddings	Ollama + nomic-embed-text	Convert text and questions into vectors
Vector database	ChromaDB	Store and retrieve document chunks
Generation	Ollama + Llama 3.2	Generate answers from retrieved context
Containerization	Docker	Package the application for reproducible deployment

Main features

- Upload a PDF and extract readable text page by page.
- Split extracted text into overlapping chunks.
- Generate vector embeddings for every chunk.
- Store chunks, embeddings, filename, and page metadata in ChromaDB.
- Convert a user's question into an embedding and retrieve the most relevant chunks.
- Use Llama 3.2 to generate an answer from the retrieved document context.
- Return source filename and page numbers with the answer.

How the RAG Pipeline Works

1. PDF upload

The user uploads a PDF through the React frontend. The frontend sends the file to the FastAPI **/upload** endpoint. The backend saves the file and passes its path to the PDF processing function.

2. Text extraction

PyPDF reads the PDF page by page. For each page, the system calls **page.extract_text()**. The extracted text is associated with the page number so that the system can later identify where retrieved information came from.

3. Chunking

The extracted text is divided into chunks of approximately 500 characters with a 50-character overlap. The overlap helps preserve context when information crosses a chunk boundary. Each chunk also receives its corresponding PDF page number.

4. Embedding generation

Each text chunk is passed to the local **nomic-embed-text** embedding model through Ollama. The model converts each chunk into a numerical vector representing its semantic meaning.

5. ChromaDB storage

The chunks, embeddings, and metadata are stored in ChromaDB. Metadata includes the original filename and the page number. This allows the system to retrieve both the content and its source location.

6. Question processing

When a user asks a question, the question is also converted into an embedding using the same embedding model. ChromaDB compares the question vector with stored chunk vectors and returns the top three most relevant chunks for the selected filename.

7. Answer generation

The retrieved chunks are passed to Llama 3.2 along with the user's question. The generation step is designed to answer using the retrieved document context rather than inventing information that is not supported by the document.

8. Sources

The backend returns the generated answer together with the retrieved metadata. The frontend can therefore show the PDF filename and page numbers used as sources.

Architecture, Current Capabilities & Roadmap

System architecture

Component	Responsibility
React frontend	Upload PDFs, send questions, display answers and sources
FastAPI	Expose /upload and /ask APIs and coordinate the RAG workflow
PyPDF	Extract text from PDF pages
Ollama embedding model	Create embeddings for chunks and questions
ChromaDB	Persist vectors, chunks, and source metadata
Ollama Llama 3.2	Generate the final answer from retrieved context

Example end-to-end flow

PDF → text extraction → page-aware chunks → embeddings → ChromaDB → question embedding → top-3 retrieval → Llama 3.2 → answer + source pages

Why RAG is used

RAG allows the application to work with information contained in user-provided PDFs without fine-tuning the generation model for every document. Documents can be indexed as they are uploaded, and relevant passages can be retrieved at question time.

Current project capabilities

- Local LLM and embedding models through Ollama.
- Persistent ChromaDB vector storage.
- Filename filtering so questions are answered against the selected document.
- Page-aware metadata for source attribution.
- Retrieval-distance inspection for experimenting with relevance thresholds.
- React frontend connected to a FastAPI backend.

Planned improvements

- Dockerize the application and its supporting services.
- Deploy the application for public demonstration.
- Improve error handling for invalid PDFs and unavailable services.
- Improve relevance detection for questions outside the document.
- Handle duplicate document uploads more cleanly.
- Add a default project-overview PDF as a built-in demo document.
- Add evaluation tests to measure retrieval and answer performance.

Portfolio goal

The project demonstrates an end-to-end RAG workflow: document ingestion, page-aware chunking, embeddings, vector retrieval, grounded generation, source attribution, API development, frontend integration, and containerization.